

Bug Report — FR-01 Đăng ký tài khoản

Sinh viên	23127195
SUT	EShop (eshop-sut) — frontend-web :5173 + backend API :3000
Phát hiện bằng	Bộ automation Playwright tests/fr01-register.spec.js
Ngày	2026-08-15
Môi trường	Windows 11, Node.js 22.20.0, Playwright 1.62.1, Chromium/Firefox/WebKit

Tổng quan: 15 lỗi trên cả ba feature.

ID	Feature	Mức độ	Tóm tắt	Test phát hiện
BUG-01	FR-01	High	Luật mật khẩu bắt buộc khoảng trắng, cấm ký tự đặc biệt — ngược với hint UI và đặc tả	TC-02, TC-27
BUG-02	FR-01	High	Không validate định dạng email ở cả frontend và backend	TC-18 → TC-21
BUG-03	FR-01	Critical	Email trùng tạo được nhiều tài khoản	TC-22
BUG-04	FR-01	Critical	Mật khẩu lưu plaintext và bị trả về trong response đăng nhập	TC-26
BUG-05	FR-12*	High	<code>GET /api/admin/users</code> không kiểm tra <code>role</code>	thủ công
BUG-06	Tài liệu	Low	<code>setup_guide.md</code> ghi sai mật khẩu admin	thủ công
BUG-07	FR-09	Critical	Công thức giảm giá phần trăm bị đảo — khách bị tính gấp 10 lần	TC-01→03, TC-18
BUG-08	FR-09	Medium	Ngưỡng đơn tối thiểu loại trừ chính giá trị bằng ngưỡng	TC-07
BUG-09	FR-09	High	Bỏ <code>user_id</code> là lách được giới hạn số lần dùng mã	TC-16
BUG-10	FR-14	Medium	Tên danh mục rỗng/ <code>null</code> /khoảng trắng vẫn tạo được	TC-05, TC-06
BUG-11	FR-14	Critical	Người dùng thường toàn quyền tạo/sửa/xoá danh mục	TC-14 → TC-16
BUG-12	FR-14	Medium	Sửa/xoá <code>id</code> không tồn tại vẫn trả về 200 thành công	TC-10, TC-12
BUG-13	FR-14	High	Xoá danh mục đang có sản phẩm làm sản phẩm mồ côi	TC-13
BUG-14	FR-14	High	Giao diện quản lý danh mục thiếu hoàn toàn chức năng Sửa	TC-08
BUG-15	FR-13*	Medium	Dashboard nhân đôi doanh thu đơn đã giao	thủ công

* BUG-05, BUG-15 nằm ngoài ba feature được giao, phát hiện tình cờ khi đọc mã nguồn và xây helper; ghi nhận để đầy đủ.

BUG-01 — Luật mật khẩu mâu thuẫn với đặc tả và với chính hint trên UI

- **Mức độ:** High
- **Thành phần:** eshop-sut/frontend-web/src/pages/Register.jsx:15
- **Test:** TC-02 (fail), TC-27 (fail), TC-10 (bằng chứng bổ trợ)
- **Ảnh:** ../evidence/bugs/BUG-01-special-char-rejected.png

Mô tả

Trang đăng ký hiển thị hint:

"Yêu cầu: Tối thiểu 8 ký tự, có chữ hoa, chữ thường, số và ký tự đặc biệt."

và `api_specification.md` dùng `"Password123!"` làm ví dụ mật khẩu hợp lệ. Tuy nhiên regex thực thi lại là:

```
const flawedStrongPasswordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*\s)[A-Za-z\d\s]{8,
```

Hai sai lệch nghiêm trọng so với đặc tả:

1. `(?=.*\s)` — **bắt buộc phải có khoảng trắng**, điều mà không tài liệu nào yêu cầu.
2. `[A-Za-z\d\s]` — character class **chỉ cho phép chữ, số và khoảng trắng**, tức là **cấm ký tự đặc biệt** — đúng thứ mà hint bảo là bắt buộc.

Hệ quả: mật khẩu đúng đặc tả bị từ chối, còn mật khẩu yếu hơn (có khoảng trắng, không có ký tự đặc biệt) lại được chấp nhận.

Steps to reproduce

1. Mở `http://localhost:5173/register`
2. Nhập Họ Tên: `Nguyen Van A`
3. Nhập Email: `test01@eshop-test.local`
4. Nhập Mật khẩu: `Password123!`
5. Bấm **Đăng Ký**

Expected

Đăng ký thành công, điều hướng sang `/login` (mật khẩu thoả toàn bộ yêu cầu ghi trên UI).

Actual

Hiện banner đỏ, ở lại `/register` :

Mật khẩu quá yếu! Phải dài tối thiểu 8 ký tự, gồm chữ hoa, chữ thường, số và KÝ TỰ ĐẶC BI

Bảng chứng đối chứng

Mật khẩu	Có ký tự đặc biệt	Có khoảng trắng	Kết quả thực tế
Password123!	✓	✗	Bị từ chối
Password123	✗	✗	Bị từ chối
Password 123	✗	✓	Được chấp nhận

Đề xuất sửa

```
const strongPasswordRegex =  
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[A-Za-z\d]).{8,}$/;
```

BUG-02 — Không validate định dạng email

- **Mức độ:** High
- **Thành phần:** `frontend-web/src/pages/Register.jsx:47` (input `type="text"` , không có kiểm tra) + `backend/server.js:20` (`POST /api/register` không validate)
- **Test:** TC-18, TC-19, TC-20, TC-21 (đều fail)
- **Ảnh:** `../evidence/bugs/BUG-02-invalid-email-accepted.png`

Mô tả

Không có tầng nào kiểm tra định dạng email:

- Frontend khai báo `<input type="text">` thay vì `type="email"` , nên trình duyệt cũng không kiểm tra hộ; `handleSubmit` chỉ kiểm tra mật khẩu.
- Backend `POST /api/register` `INSERT` thẳng vào bảng, không validate.

Steps to reproduce

1. Mở `http://localhost:5173/register`
2. Nhập Họ Tên: `Nguyen Van A`
3. Nhập Email: `abc`
4. Nhập Mật khẩu: `Password 123`
5. Bấm **Đăng Ký**

Expected

Từ chối với thông báo email không hợp lệ; **không** tạo bản ghi trong bảng `users`.

Actual

Đăng ký thành công, điều hướng `/login`. Kiểm chứng qua `GET /api/admin/users` cho thấy đã tồn tại một dòng với `email = "abc"`.

Các giá trị sau đều được chấp nhận: `abc`, `abc@`, `@domain.com`, `a b@domain.com`.

Đề xuất sửa

Đổi input sang `type="email"`, thêm regex kiểm tra ở frontend, và **quan trọng hơn** là validate ở backend trước khi `INSERT` (client-side validation luôn có thể bị bỏ qua bằng cách gọi thẳng API).

BUG-03 — Email trùng tạo được nhiều tài khoản

- **Mức độ:** Critical
- **Thành phần:** `backend/database.js:50` (thiếu `UNIQUE`) + `backend/server.js:20` (không kiểm tra tồn tại)
- **Test:** TC-22 (fail)
- **Ảnh:** `../evidence/bugs/BUG-03-duplicate-email-rows.png`

Mô tả

Bảng `users` khai báo `email TEXT` **không có ràng buộc** `UNIQUE`:

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT,  
  email TEXT,          -- thiếu UNIQUE  
  password TEXT,  
  ...  
)
```

và `POST /api/register` không kiểm tra email đã tồn tại hay chưa. Kết quả là nhiều tài khoản có thể dùng chung một địa chỉ email.

Đây là lỗi **Critical** vì email là định danh đăng nhập: `POST /api/login` dùng `db.get(... WHERE email = ?)` nên chỉ trả về **dòng đầu tiên**. Người đăng ký sau không bao giờ đăng nhập được vào tài khoản của chính mình, và luồng "quên mật khẩu" (FR-03) trở nên không xác định.

Steps to reproduce

1. Đăng ký tài khoản với email `dup@eshop-test.local`, mật khẩu `Password 123`
2. Quay lại `/register`
3. Đăng ký **lại** đúng email đó với tên khác
4. Đăng nhập admin, gọi `GET /api/admin/users`

Expected

Bước 3 bị từ chối với thông báo "Email đã được sử dụng". Số dòng mang email đó = **1**.

Actual

Bước 3 thành công. Số dòng mang email đó = **2**:

```
[  
  { "id": 49, "name": "Nguoi Dung Goc", "email": "bug03...@eshop-test.local", "role": "  
  { "id": 50, "name": "Nguoi Dung Trung", "email": "bug03...@eshop-test.local", "role": "  
]
```

Đề xuất sửa

Thêm `UNIQUE` cho cột `email` **và** kiểm tra tồn tại trong `POST /api/register`, trả về HTTP 409 kèm thông báo rõ ràng.

BUG-04 — Mật khẩu lưu plaintext và bị trả về trong response đăng nhập

- **Mức độ:** Critical
- **Thành phần:** backend/server.js:20 (lưu) và backend/server.js:52 (trả về)
- **Test:** TC-26 (fail)
- **Ảnh:** ../evidence/bugs/BUG-04-plaintext-password.png

Mô tả

Hai vấn đề chồng lên nhau:

1. POST /api/register INSERT thẳng mật khẩu thô, không hash:

```
db.run("INSERT INTO users (name, email, password) VALUES (?, ?, ?)", [name, email, pas
```

2. POST /api/login SELECT * rồi trả nguyên đối tượng user về client:

```
res.json({ message: "Login successful", token, user });
```

nên response chứa luôn trường password .

Steps to reproduce

1. POST /api/register với {"name":"Plain Text","email":"plain@eshop-test.local","password":"Password 123"}
2. POST /api/login với {"email":"plain@eshop-test.local","password":"Password 123"}
3. Đọc trường user.password trong response

Expected

Mật khẩu được hash (bcrypt/argon2) khi lưu, và **không bao giờ** xuất hiện trong bất kỳ response nào.

Actual

```
Password sent at registration : "Password 123"
Password echoed by the API    : "Password 123"
Identical                    : true
```

Bất kỳ ai xem được response (DevTools, log proxy, biến thể XSS) đều lấy được mật khẩu thô. Nếu database rò rỉ, toàn bộ mật khẩu lộ ngay lập tức.

Đề xuất sửa

Hash bằng `bcrypt` khi đăng ký, so sánh bằng `bcrypt.compare` khi đăng nhập, và loại bỏ trường `password` khỏi mọi payload trả về.

BUG-05 — `GET /api/admin/users` không kiểm tra quyền

- **Mức độ:** High
- **Thành phần:** `backend/server.js:100` (`authenticateToken`) và `server.js:494`
- **Phạm vi:** thuộc **FR-12 (Access control)**, nằm ngoài FR-01 — ghi nhận vì phát hiện được trong quá trình xây bộ test FR-01
- **Test:** không có test riêng; xác minh thủ công

Mô tả

Middleware `authenticateToken` chỉ xác minh chữ ký JWT, **không kiểm tra** `req.user.role` :

```
const authenticateToken = (req, res, next) => {
  const token = authHeader && authHeader.split(" ")[1];
  if (token == null) return res.status(401).json({ error: "Unauthorized" });
  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(403).json({ error: "Forbidden" });
    req.user = user;
    next();           // <-- không hề kiểm tra role
  });
};
```

Endpoint `/api/admin/users` chỉ gắn middleware này, nên **bất kỳ tài khoản thường nào cũng đọc được toàn bộ danh sách người dùng**.

Steps to reproduce

1. Đăng ký một tài khoản thường bất kỳ
2. Đăng nhập, lấy `token` (JWT có `role: "user"`)
3. `GET /api/admin/users` với header `Authorization: Bearer <token>`

Expected

HTTP 403 Forbidden.

Actual

HTTP 200, trả về toàn bộ bảng người dùng:

```
logged in as role=user  
ADMIN ENDPOINT ACCESSIBLE by role='user' -> returned 52 user rows
```

Đề xuất sửa

Thêm middleware `requireAdmin` kiểm tra `req.user.role === 'admin'` cho toàn bộ nhóm route `/api/admin/*`.

BUG-06 — `setup_guide.md` ghi sai mật khẩu admin

- **Mức độ:** Low (lỗi tài liệu)
- **Thành phần:** `eshop-sut/setup_guide.md:105` vs `eshop-sut/backend/database.js:92`

Mô tả

Tài liệu hướng dẫn cài đặt ghi tài khoản admin mặc định là:

Email: `admin@eshop.com` — **Mật khẩu:** `admin123`

nhưng script seed thực tế lại ghi:

```
insertUser.run('Admin User', 'admin@eshop.com', 'Admin123!', 'admin');
```

Tác động

Người dùng làm theo tài liệu sẽ đăng nhập sai. Do FR-02 có lỗi cộng dồn `login_attempts + 2` với ngưỡng khoá là 3, **chỉ cần 2 lần thử theo tài liệu là tài khoản admin bị khoá 180 giây** — và trong thời gian khoá, ngay cả mật khẩu đúng cũng bị từ chối vì hệ thống kiểm tra khoá **trước** khi so mật khẩu.

Lỗi tài liệu này đã trực tiếp làm hỏng 5 test trong lần chạy đầu tiên của bộ automation (xem R-08 trong báo cáo chính).

Đề xuất sửa

Sửa `setup_guide.md` thành `Admin123!`, hoặc đổi seed cho khớp tài liệu.

BUG-07 — Công thức giảm giá phần trăm bị đảo, khách bị tính gấp 10 lần

- **Mức độ:** Critical — đây là lỗi nghiêm trọng nhất của toàn bài
- **Thành phần:** `backend/server.js:399` và `server.js:419`
- **Test:** TC-01, TC-02, TC-03, TC-18 (đều fail)
- **Ảnh:** `../evidence/bugs/BUG-07-percent-coupon-overcharge.png`, `BUG-07-percent-coupon-api.png`

Mô tả

Với mã loại `percent`, `discount_value` lưu số phần trăm (ví dụ `SAVE10` có `discount_value = 10`). Nhưng công thức tính lại là:

```
discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));
```

Thay vì `total_amount * discount_value / 100`. Với `discount_value = 10`, biểu thức thành `total * (1 - 10) = total * (-9) → số tiền giảm âm`. Vì `final_amount = total_amount - discount_amount`, trừ một số âm thành cộng:

```
final = total - (-9 * total) = 10 * total
```

Steps to reproduce

1. Đăng nhập, vào `http://localhost:5173/checkout`
2. Đặt "Tổng tiền thanh toán" = `500000`
3. Nhập mã `SAVE10`, bấm **Áp dụng**

Expected

Giảm 50.000 đ (10% của 500.000), thành tiền 450.000 đ.

Actual

✅ Áp dụng thành công! Giảm 10%
Tiết kiệm: -4,500,000 đ
Thành tiền: 5,000,000 đ

Khách bị tính **gấp 10 lần** giá trị đơn hàng, trong khi giao diện vẫn hiển thị dấu ✅ và câu "Áp dụng thành công! Giảm 10%". Người dùng không có cách nào nhận ra ngoài việc tự đọc con số.

Kiểm chứng qua API:

```
POST /api/apply-coupon {"code":"SAVE10","total_amount":500000}
{"success":true,"coupon_id":1,"discount_amount":-4500000,"final_amount":5000000,...}
```

Lỗi này phá vỡ hai bất biến cơ bản mà TC-18 kiểm tra: **số tiền giảm không bao giờ được âm**, và **thành tiền không bao giờ được lớn hơn tổng gốc**.

Đề xuất sửa

```
discount_amount = coupon.type === 'percent'
  ? Math.floor(total_amount * coupon.discount_value / 100)
  : coupon.discount_value;
discount_amount = Math.min(discount_amount, total_amount); // không giảm quá tổng đơn
```

BUG-08 — Ngưỡng đơn tối thiểu loại trừ chính giá trị bằng ngưỡng

- **Mức độ:** Medium (lỗi biên)
- **Thành phần:** backend/server.js:379
- **Test:** TC-07 (fail)
- **Ảnh:** ../evidence/bugs/BUG-08-min-order-boundary.png

Mô tả

```
if (total_amount > coupon.min_order_amount) {
```

Thông báo lỗi của chính hệ thống ghi "Đơn hàng chưa đủ giá trị **tối thiểu** 500,000 đ" — "tối thiểu" nghĩa là `>=`. Nhưng điều kiện dùng `>`, nên đơn đúng bằng ngưỡng bị từ chối.

Steps to reproduce

total_amount	Kết quả
500.000 (= ngưỡng của BIGBUY)	❌ HTTP 400 — "chưa đủ giá trị tối thiểu 500,000 đ"
500.001	✅ HTTP 200, giảm 50.000 đ

Expected / Actual

Đơn 500.000 đ phải được áp mã (bằng ngưỡng tối thiểu). Thực tế bị từ chối, và thông báo lỗi tự mâu thuẫn: nó nói ngưỡng là 500.000 trong khi từ chối đúng 500.000.

Đề xuất sửa

Đổi `>` thành `>=`.

BUG-09 — Bỏ `user_id` là lách được giới hạn số lần dùng mã

- **Mức độ:** High
- **Thành phần:** `backend/server.js:386` (`if (user_id) { ... }`)
- **Test:** TC-16 (fail)
- **Ảnh:** `../evidence/bugs/BUG-09-usage-limit-bypass.png`

Mô tả

Kiểm tra `max_uses_per_user` chỉ nằm **bên trong** nhánh `if (user_id)`. Nhánh `else` tính giảm giá và trả về thành công mà không hề tra bảng `coupon_usage`. Vì `user_id` do client gửi lên, chỉ cần bỏ trường này là bỏ qua toàn bộ giới hạn.

Steps to reproduce

1. Đăng ký một tài khoản, dùng hết quota của `VIP100` (`max_uses_per_user = 2`)
2. Gọi `POST /api/apply-coupon` **có** `user_id` → HTTP 400 "đã đạt giới hạn"
3. Gọi lại **không** có `user_id`, mọi thứ khác giữ nguyên

Expected

Bước 3 vẫn bị từ chối — giới hạn gắn với tài khoản, không phụ thuộc client có khai báo danh tính hay không.

Actual

```
WITH user_id      -> HTTP 400 {"error":"Bạn đã sử dụng mã này 2 lần (đã đạt giới hạn)"}
WITHOUT user_id   -> HTTP 200 {"success":true,"discount_amount":100000,...}
```

Mã có thể tái sử dụng vô hạn.

Đề xuất sửa

Lấy danh tính từ JWT (`req.user.id`) thay vì tin `user_id` trong body, và bắt buộc xác thực cho endpoint này.

BUG-10 — Tên danh mục không được validate

- **Mức độ:** Medium
- **Thành phần:** `backend/server.js:249`
- **Test:** TC-05, TC-06 (fail)
- **Ảnh:** `../evidence/bugs/BUG-10-blank-category-name.png`

Mô tả

`POST /api/categories` `INSERT` thẳng giá trị `name` nhận được, không kiểm tra rỗng, không kiểm tra thiếu trường, không `trim()`.

Steps to reproduce

1. Vào Web Admin → **Danh mục**, để trống ô "Tên danh mục mới", bấm **Thêm mới**
2. Hoặc gọi `POST /api/categories` với body `{}`

Expected

HTTP 400, không tạo bản ghi.

Actual

```
POST /api/categories {"name":""} -> HTTP 200 {"message":"Category created","id":4}
POST /api/categories {} -> HTTP 200 {"message":"Category created","id":5}
```

Bảng danh mục xuất hiện dòng tên rỗng và dòng tên `null`. Trên giao diện chúng hiện thành ô trắng không thể phân biệt, và vẫn gán được cho sản phẩm.

Đề xuất sửa

Validate `typeof name === 'string' && name.trim().length > 0` trước khi `INSERT`.

BUG-11 — Người dùng thường toàn quyền quản lý danh mục

- **Mức độ:** Critical
- **Thành phần:** `backend/server.js:100` (`authenticateToken`), áp cho `server.js:249/257/269`
- **Test:** TC-14, TC-15, TC-16 (đều fail)
- **Ảnh:** `../evidence/bugs/BUG-11-category-access-control.png`

Mô tả

Cả ba endpoint ghi của danh mục chỉ gán `authenticateToken`, mà middleware này **chỉ xác minh chữ ký JWT**, không đọc `req.user.role`. Bất kỳ khách hàng nào đăng ký xong đều có toàn quyền CRUD trên danh mục sản phẩm của toàn hệ thống.

Steps to reproduce

1. Đăng ký một tài khoản thường bất kỳ, đăng nhập lấy `token`
2. `POST /api/categories` với `Authorization: Bearer <token>`
3. `DELETE /api/categories/:id` với cùng token

Expected

HTTP 403 Forbidden ở cả hai bước.

Actual

```
Account role: user
POST    /api/categories      -> HTTP 200  {"message":"Category created","id":...}
DELETE  /api/categories/7    -> HTTP 200  {"message":"Category deleted"}
```

Kết hợp với BUG-13, một tài khoản thường có thể xoá sạch danh mục và làm mờ coi toàn bộ sản phẩm của cửa hàng.

Đề xuất sửa

Thêm middleware `requireAdmin` kiểm tra `req.user.role === 'admin'` cho toàn bộ endpoint ghi của danh mục (và các endpoint quản trị khác — xem BUG-05).

BUG-12 — Sửa/xoá `id` không tồn tại vẫn báo thành công

- **Mức độ:** Medium
- **Thành phần:** `backend/server.js:257` (PUT), `server.js:269` (DELETE)
- **Test:** TC-10, TC-12 (fail)
- **Ảnh:** `../evidence/bugs/BUG-12-nonexistent-id-reports-success.png`

Mô tả

Cả hai handler đều bỏ qua `this.changes` — số dòng thực sự bị tác động — nên luôn trả về 200 kèm thông báo thành công dù không đụng vào dòng nào.

Steps to reproduce

```
PUT    /api/categories/999999  {"name":"Ghost"}
DELETE /api/categories/999999
```

Expected

HTTP 404 Not Found cho cả hai.

Actual

```
PUT    /api/categories/999999 -> HTTP 200 {"message":"Category updated"}
DELETE /api/categories/999999 -> HTTP 200 {"message":"Category deleted"}
```

Client không có cách nào phân biệt "đã sửa thành công" với "không tìm thấy", dẫn tới giao diện báo thành công giả.

Đề xuất sửa

```
if (this.changes === 0) return res.status(404).json({ error: "Category not found" });
```

BUG-13 — Xoá danh mục đang có sản phẩm làm sản phẩm mồ côi

- **Mức độ:** High
- **Thành phần:** backend/server.js:269 + database.js (thiếu khoá ngoại)
- **Test:** TC-13 (fail)
- **Ảnh:** ../evidence/bugs/BUG-13-delete-category-orphans-products.png

Mô tả

Bảng `products` có cột `category_id` nhưng **không khai báo** `FOREIGN KEY`, và handler `DELETE` không kiểm tra xem còn sản phẩm nào tham chiếu hay không.

Steps to reproduce

1. Tạo danh mục mới, ghi lại `id`
2. Tạo một sản phẩm với `category_id` là `id` đó
3. Xoá danh mục vừa tạo
4. Gọi `GET /api/products`

Expected

Bước 3 bị từ chối (HTTP 409) kèm thông báo danh mục đang được sử dụng — hoặc phải xử lý sản phẩm liên quan trước.

Actual

```
DELETE /api/categories/8 -> HTTP 200 {"message":"Category deleted"}
Products still pointing at the deleted category: 1
```

Sản phẩm giữ `category_id` trở tới danh mục không còn tồn tại. Trên giao diện admin, ô danh mục của sản phẩm đó trở thành rỗng và không sửa được về trạng thái hợp lệ nếu không gán lại thủ công.

Đề xuất sửa

Khai báo `FOREIGN KEY (category_id) REFERENCES categories(id)`, bật `PRAGMA foreign_keys = ON`, và kiểm tra số sản phẩm liên quan trước khi xoá.

BUG-14 — Giao diện quản lý danh mục thiếu hoàn toàn chức năng Sửa

- **Mức độ:** High (thiếu chức năng so với đặc tả)
- **Thành phần:** `frontend-admin/src/App.jsx:294-335`
- **Test:** TC-08 (fail)
- **Ảnh:** `../evidence/bugs/BUG-14-no-edit-control.png`

Mô tả

FR-14 được đặc tả là **Category management (CRUD)** và backend đã có sẵn `PUT /api/categories/:id`. Nhưng màn hình quản lý danh mục chỉ render đúng một nút trên mỗi dòng:

```
<button ... onClick={() => deleteCategory(c.id)}>Xóa</button>
```

Không có nút Sửa, không có ô input inline, không có modal. Chữ **U** trong CRUD không thể thực hiện được từ giao diện — endpoint `PUT` tồn tại nhưng không có đường nào gọi tới.

Steps to reproduce

1. Đăng nhập Web Admin (`http://localhost:5174`) bằng `admin@eshop.com` / `Admin123!`
2. Mở tab **Danh mục**
3. Quan sát cột **Hành động** của mọi dòng

Expected

Có nút/điều khiển cho phép đổi tên danh mục.

Actual

Tự động liệt kê toàn bộ control trên một dòng cho kết quả:

```
row actions offered by the admin UI = ["Xóa"]
```

Ngoài ra nút **Xóa** không có hộp thoại xác nhận — một cú nhấp nhầm là mất danh mục ngay (kết hợp BUG-13 thì mất luôn liên kết của sản phẩm).

Đề xuất sửa

Bổ sung nút "Sửa" mở form đổi tên gọi `PUT /api/categories/:id`, và thêm xác nhận trước khi xóa.

BUG-15 — Dashboard nhân đôi doanh thu đơn đã giao

- **Mức độ:** Medium
- **Thành phần:** `frontend-admin/src/App.jsx:218`
- **Phạm vi:** thuộc **FR-13 (Dashboard)**, ngoài ba feature được giao — ghi nhận vì phát hiện khi đọc mã nguồn cho FR-14
- **Test:** không có test tự động; xác minh bằng đọc mã

Mô tả

```
const totalRevenue = orders.reduce((sum, o) => {  
  if (o.status === "delivered") return sum + o.total_amount * 2;  
  return sum;  
}, 0);
```

Doanh thu của mỗi đơn `delivered` được nhân 2 không có lý do nghiệp vụ nào. Số liệu tổng doanh thu trên dashboard luôn gấp đôi giá trị thật.

Đề xuất sửa

Bỏ `* 2`.

Trạng thái GitHub Issues

Toàn bộ 15 bug đã được đăng lên GitHub Issues của repository, mỗi issue kèm ảnh chụp bằng chứng:

Bug	Issue
BUG-01	#1
BUG-02	#2
BUG-03	#3
BUG-04	#4
BUG-05	#5
BUG-06	#6
BUG-07	#7
BUG-08	#8
BUG-09	#9
BUG-10	#10
BUG-11	#11
BUG-12	#12
BUG-13	#13
BUG-14	#14
BUG-15	#15

Ảnh bằng chứng gốc nằm ở `23127195/evidence/bugs/` .